



UNIVERSIDADE ESTADUAL DE MARINGÁ
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

EDUARDO ANGELO ROZADA MINHOLI – RA 134932
NUNO MIGUEL MENDONÇA ABILIO – RA 132830

RELATÓRIO TÉCNICO DO TRABALHO 2 DE PDI 2026

MÁSCARA DE NITIDEZ E FILTRAGEM HIGH-BOOST

Universidade Estadual de Maringá – Departamento de Informática
Av. Colombo, 5790 – Bloco C56, Zona 7 – CEP: 87020-900
Fone: (44) 3011-4324/3011-4219
Maringá – PR
2026

EDUARDO ANGELO ROZADA MINHOLI – RA 134932
NUNO MIGUEL MENDONÇA ABILIO – RA 132830

MÁSCARA DE NITIDEZ E FILTRAGEM HIGH-BOOST

Relatório Técnico do Trabalho 2 apresentado ao Departamento de Informática da Universidade Estadual de Maringá como requisito parcial à aprovação na disciplina de Processamento Digital de Imagens do Curso de Bacharelado em Ciência da Computação.

Prof. Dr. Franklin Cesar Flores.

Universidade Estadual de Maringá – Departamento de Informática
Av. Colombo, 5790 – Bloco C56, Zona 7 – CEP: 87020-900
Fone: (44) 3011-4324/3011-4219
Maringá – PR
2026

Sumário

	Páginas
1 Objetivo do Trabalho	4
2 Fundamentação Teórica	4
3 Código-Fonte	5
4 Casos de Teste e Análise de Resultados	7
4.1 Caso de Controle: Validação Matemática da Identidade ($k = 0.0$)	8
4.2 Análise do Impacto do Fator de Ganho (k)	8
4.3 Análise do Impacto da Dimensão da Máscara de Suavização (m)	9
4.4 Comportamento em Imagens com Detalhes Finos (Análise de Texto)	11
4.5 Cenário Extremo de Estresse ($m = 51$ e $k = 100.0$)	12
5 Conclusão	13

1 Objetivo do Trabalho

O objetivo deste trabalho é implementar e testar um programa, desenvolvido em Python 3.x, que receba uma imagem em tons de cinza e dois parâmetros numéricos: m (dimensão de um filtro de média simples de tamanho $m \times m$) e k (fator de ganho multiplicativo). O programa deve retornar duas imagens de saída: a máscara de nitidez obtida pelo processo e a imagem aguçada resultante, gerada pela aplicação da técnica de Máscara de Nitidez clássica ou pela Filtragem High-Boost, conforme as especificações descritas na Seção 3.6.3 da literatura de Gonzalez-Woods (3ª edição).

2 Fundamentação Teórica

O processo de realce de bordas por meio de filtragem espacial baseia-se na atenuação das componentes de baixa frequência (borramento) de uma imagem para destacar suas transições abruptas de intensidade. O método abordado neste trabalho é estruturado em três etapas matemáticas fundamentais:

1. **Borrimento da Imagem Original:** Produz-se uma versão suavizada da imagem, denotada por $\bar{f}(x,y)$, por meio da convolução com um filtro linear da média simples de dimensões $m \times m$. A escolha de m deve ser restrita a inteiros ímpares e positivos para garantir a existência de um pixel central bem definido:

$$\bar{f}(x,y) = f(x,y) * w_{m \times m} \quad (1)$$

Onde w representa a máscara de média com coeficientes equivalentes a $1/m^2$.

2. **Obtenção da Máscara de Nitidez:** A máscara, denotada por $g_{mask}(x,y)$, consiste em um sinal de alta frequência contendo apenas os detalhes e transições de borda, calculada através da subtração da imagem borrada da imagem original:

$$g_{mask}(x,y) = f(x,y) - \bar{f}(x,y) \quad (2)$$

Visto que essa operação pode resultar em valores de intensidade negativos (onde a imagem original é mais escura que a média local), o processamento numérico deve ser realizado obrigatoriamente utilizando tipos de dados de ponto flutuante (float64) para evitar estouros de memória por subfluxo (*underflow*).

3. **Filtragem High-Boost:** A imagem final aguçada, $g(x,y)$, é obtida somando-se a máscara ponderada pelo fator de ganho k de volta à imagem original:

$$g(x,y) = f(x,y) + k \cdot g_{mask}(x,y) \quad (3)$$

O comportamento do algoritmo varia de acordo com o parâmetro k :

- Se $k = 0$, o resultado é estritamente a imagem original.
- Se $k = 1$, o método reduz-se ao algoritmo tradicional de **Máscara de Nitidez** (*Unsharp Masking*).
- Se $k > 1$, o processo é denominado **Filtragem High-Boost**, em que a contribuição dos detalhes é amplificada, sendo ideal para recuperar imagens com alto grau de borramento original.

Após o cálculo, os valores resultantes em $g(x,y)$ podem extrapolar os limites de exibição física de luminância. Portanto, aplica-se uma operação de saturação (*clipping*) para restringir os valores ao intervalo dinâmico $[0, 255]$ antes da conversão para inteiros sem sinal de 8 bits (uint8). Para fins de visualização da máscara $g_{mask}(x,y)$, realiza-se uma normalização linear que mapeia uniformemente o menor valor para 0 e o maior valor para 255.

3 Código-Fonte

O código a seguir apresenta a implementação completa do programa utilizando as bibliotecas scikit-image, numpy e o operador de convolução bidimensional da biblioteca scipy.

```
1 import sys
2 import numpy as np
3 import skimage as ski
4 from scipy.ndimage import convolve
5 from os.path import splitext
6 import os
7
8 def mascara_nitidez_high_boost(img_atalho: str, m: int, k: float):
9     '''Recebe uma imagem em tons de cinza e dois parâmetros m e k e
10         devolve uma outra imagem em
11         tons de cinza aguçada pela aplicação de máscara de nitidez ou
12         filtragem high-boost.'''
13
14     imagem_original = ski.io.imread(img_atalho)
15
16     # Antes precisa garantir que a imagem esteja em tons de cinza
17     if imagem_original.ndim == 3:
18         imagem_original = ski.color.rgb2gray(imagem_original) * 255
19
20     # e aqui podem aparecer negativos na máscara, mas precisa
21     # converter para float
22     # para evitar erros de estouro na subtração
```

```

19     imagem_original = imagem_original.astype(np.float64)
20
21
22     # Borra a imagem original com um filtro da média simples m x m
    usando a função pronta de convolução
23     filtro_media = np.ones((m, m), dtype=np.float64) / (m * m)
24     imagem_borrada = convolve(imagem_original, filtro_media, mode="
        reflect")
25
26     # Máscara de nitidez criada pela diferença entre a imagem
    original e a borrada
27     mascara = imagem_original - imagem_borrada
28
29     # Filtragem high-boost
30     imagem_agucada = imagem_original + k * mascara
31
32     # Satura os valores fora do intervalo [0, 255] antes de
    converter para uint8
33     imagem_agucada = np.clip(imagem_agucada, 0, 255).astype(np.uint
        8)
34
35     # Normaliza a máscara apenas para fins de visualização (pq ela
    possui valores negativos)
36     mascara_normalizada = normalizar_para_visualizacao(mascara)
37
38     os.makedirs("figure", exist_ok=True)
39
40     nome_base, _ = splitext(img_atalho)
41
42     ski.io.imsave(f"figure/{nome_base}_m={m}_k={k}_mascara.jpg",
        mascara_normalizada)
43     ski.io.imsave(f"figure/{nome_base}_m={m}_k={k}_agucada.jpg",
        imagem_agucada)
44
45
46 def normalizar_para_visualizacao(matriz: np.ndarray) -> np.ndarray:
47     '''Normaliza uma matriz com valores quaisquer (negativos também
        ) para o intervalo [0, 255],
48     para que possa ser visualizada como uma imagem.'''
49     minimo = matriz.min()
50     maximo = matriz.max()

```

```

51     if maximo - minimo == 0: # evita divisão por zero (imagem
        constante)
52         return np.zeros_like(matriz, dtype=np.uint8)
53     matriz_normalizada = (matriz - minimo) / (maximo - minimo) * 25
        5
54     return matriz_normalizada.astype(np.uint8)
55
56
57 if __name__ == "__main__":
58     if len(sys.argv) < 4:
59         print("Erro! Modo de uso: python main.py <nome_imagem> <
        valor_m> <valor_k>")
60         sys.exit(1)
61
62     img_atalho = sys.argv[1]
63     m = int(sys.argv[2])
64     k = float(sys.argv[3])
65
66     if m < 1 or m % 2 == 0:
67         print("Erro! O parâmetro m deve ser um número inteiro ímpar
        e positivo (ex.: 3, 5, 7...)")
68         sys.exit(1)
69
70     if k < 0:
71         print("Erro! O parâmetro k deve ser maior ou igual a 0")
72         sys.exit(1)
73
74     mascara_nitidez_high_boost(img_atalho, m, k)

```

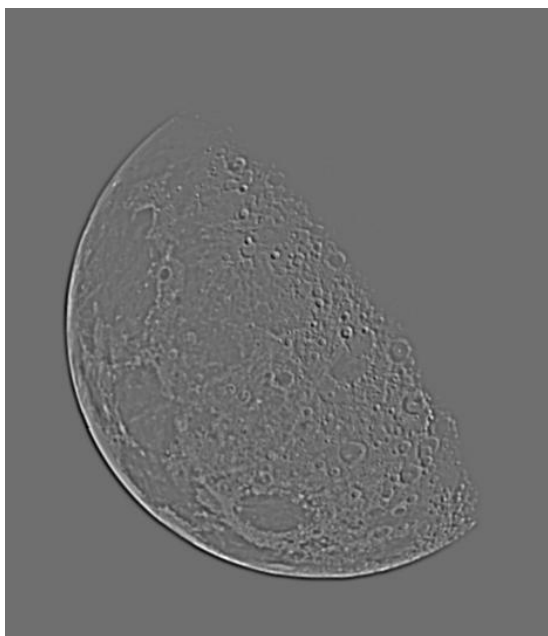
Listing 1. Implementação da filtragem High-Boost e máscara de nitidez

4 Casos de Teste e Análise de Resultados

Para validar e documentar o comportamento do algoritmo sob diferentes condições operacionais, foram feitos diversos testes, variando a dimensão do filtro (m) e o ganho multiplicativo (k). Os testes foram aplicados em duas imagens de naturezas distintas disponibilizadas no livro-texto: `blurry_moon.tif` (Imagem 3.38-a, caracterizada por transições suaves de baixa frequência e iluminação difusa) e `dipxe_text` (Imagem 3.40-a, contendo estruturas de altíssima frequência representadas por caracteres alfanuméricos em texto).

4.1 Caso de Controle: Validação Matemática da Identidade ($k = 0.0$)

O primeiro teste serve para checar se o código funciona de acordo com a teoria. Quando definimos $k = 0.0$, a parte da fórmula que soma a máscara de nitidez é zerada, então o resultado precisa ser igual à imagem original.



(a) Máscara normalizada ($m = 7, k = 0.0$).



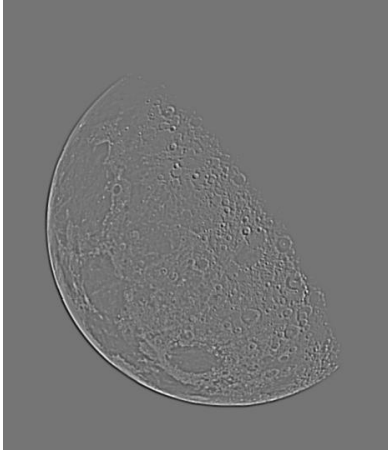
(b) Imagem resultante ($m = 7, k = 0.0$).

Figura 1. Caso de controle para a imagem da Lua. Observa-se que a imagem processada permanece idêntica à imagem de entrada original devido ao ganho nulo.

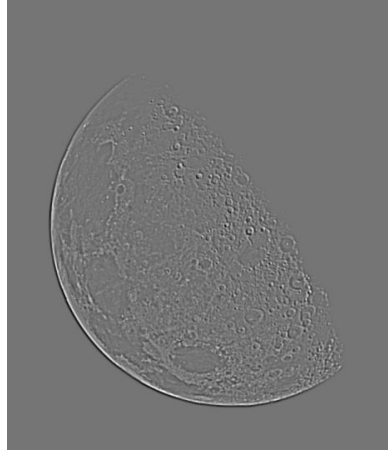
Análise: Como podemos ver na Figura 1, a imagem de saída não teve nenhuma mudança visual em comparação com a original. A máscara criada com $m = 7$ destaca bem as bordas e crateras da Lua, mas como foi multiplicada por zero, o programa manteve a imagem original intacta, mostrando que a lógica matemática do código está certa.

4.2 Análise do Impacto do Fator de Ganho (k)

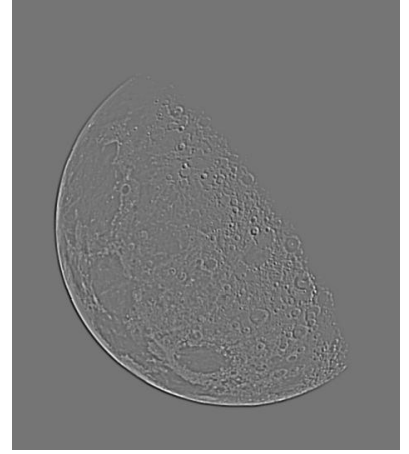
Deixando o tamanho do filtro fixo em $m = 5$, testamos como a nitidez aumenta na imagem da Lua usando três valores de ganho: a Máscara de Nitidez comum ($k = 1.0$), a Filtragem High-Boost moderada ($k = 3.0$) e um ganho muito alto que estoura os pixels ($k = 30.0$).



(a) *Máscara* ($m = 5, k = 1.0$).



(b) *Máscara* ($m = 5, k = 3.0$).



(c) *Máscara* ($m = 5, k = 30.0$).



(d) *Resultado* ($m = 5, k = 1.0$).



(e) *Resultado* ($m = 5, k = 3.0$).



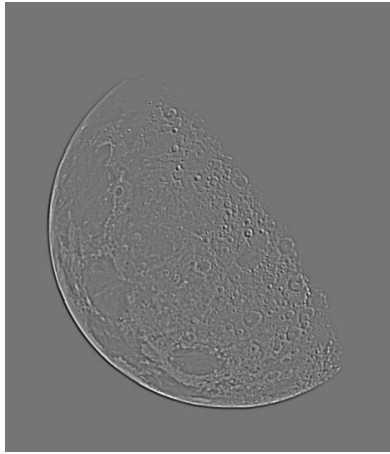
(f) *Resultado* ($m = 5, k = 30.0$).

Figura 2. *Comparativo de variação do fator de ganho k sob máscara constante de 5×5 na imagem da Lua.*

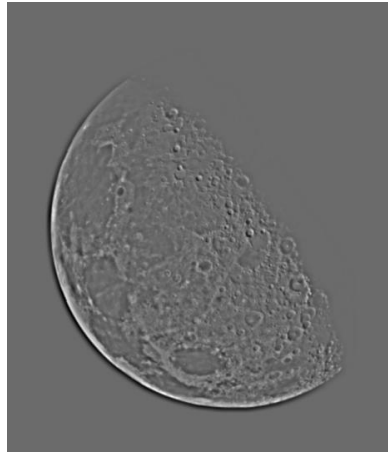
Análise: Na Figura 2, dá para notar que com $k = 1.0$ o resultado melhora um pouco a nitidez, deixando algumas crateras pequenas mais visíveis. Quando aumentamos para $k = 3.0$, o efeito melhora bastante e os contornos da Lua ficam bem mais destacados, sem estragar o brilho geral da imagem. Porém, com o ganho exagerado em $k = 30.0$, a imagem fica muito distorcida e feia. Os tons de cinza viram preto ou branco direto e aparecem manchas e linhas artificiais. Isso acontece porque os valores passam de 255 e o comando `np.clip` corta os excessos, provando que valores muito altos de ganho estragam a imagem.

4.3 Análise do Impacto da Dimensão da Máscara de Suavização (m)

Para ver como o tamanho do borrão muda o resultado, deixamos o ganho fixo em $k = 3.0$ e aumentamos o tamanho do filtro de média para $m = 5$, $m = 13$ e $m = 31$ na imagem da Lua.



(a) Máscara ($m = 5$).



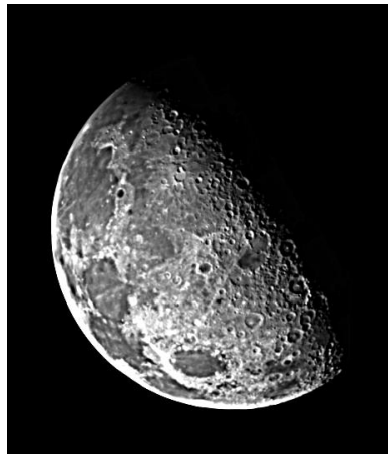
(b) Máscara ($m = 13$).



(c) Máscara ($m = 31$).



(d) Resultado ($m = 5$).



(e) Resultado ($m = 13$).



(f) Resultado ($m = 31$).

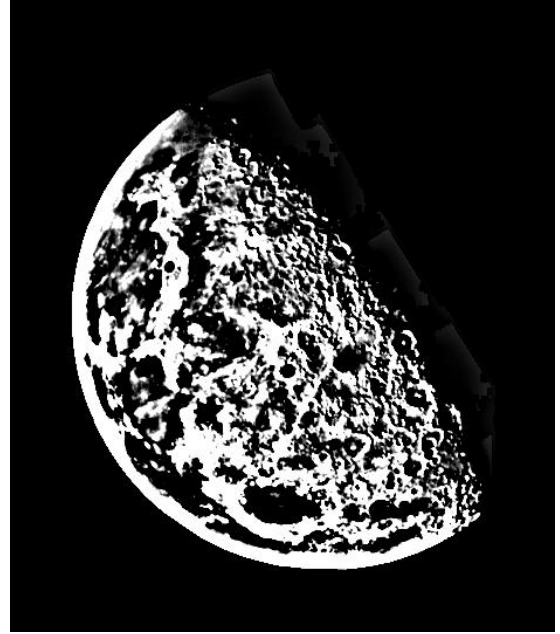
Figura 3. Comparativo de variação do tamanho da máscara m sob ganho constante de $k = 3.0$ na imagem da Lua.

Análise: Olhando as máscaras na Figura 3, percebemos que quanto maior o valor de m , mais borrada fica a imagem intermediária. Por causa disso, a máscara começa a pegar não só os detalhes finos, mas também áreas maiores da imagem. No resultado com $m = 31$, dá para ver uma espécie de "sombra" ou contorno cinza bem grosso em volta da Lua. Isso mostra que filtros muito grandes deixam o efeito de nitidez grosso e artificial nas bordas.

Para ver o que acontece juntando o pior dos dois mundos (máscara gigante e ganho muito alto), rodamos o teste com $m = 31$ e $k = 30.0$ na Figura 4.



(a) *Máscara extrema ($m = 31, k = 30.0$).*



(b) *Resultado saturado ($m = 31, k = 30.0$).*

Figura 4. *Cenário de estresse paramétrico com máxima espessura de borda e máximo ganho na imagem da Lua.*

4.4 Comportamento em Imagens com Detalhes Finos (Análise de Texto)

Mudar os testes para a imagem de texto `dipxe_text` traz resultados bem diferentes. Como o texto é feito de letras pequenas com traços finos (preto no branco), qualquer erro na escolha dos parâmetros estraga a leitura do texto muito mais fácil do que estragaria o desenho da Lua.



(a) *Resultado ($m = 5, k = 1.0$).*



(b) *Resultado ($m = 5, k = 3.0$).*



(c) *Resultado ($m = 5, k = 30.0$).*

Figura 5. *Resultados obtidos para a imagem de texto variando-se o ganho k com tamanho de filtro fixado em $m = 5$.*

Análise: No teste mudando o ganho (Figura 5), o ajuste padrão ($k = 1.0$) funciona muito bem e tira o borrado das letras originais. Com $k = 3.0$, começam a aparecer umas sombras escuras em volta das letras claras por causa do contorno forte da máscara. Já com o ganho em $k = 30.0$, o texto não chega a ficar ilegível, o fundo ganha uma forte borda preta, deformando um pouquinho as letras.

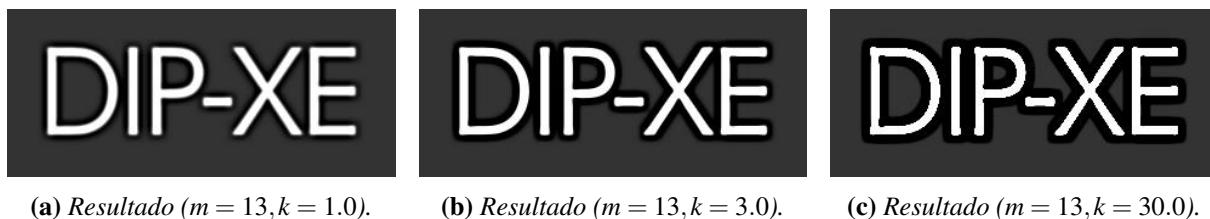


Figura 6. Resultados obtidos para a imagem de texto expandindo a dimensão da janela do filtro para $m = 13$.

Análise: Quando aumentamos o tamanho da máscara para $m = 13$ (Figura 6), o borrão do filtro fica maior do que a própria espessura das letras. Isso faz com que o realce se espalhe demais. Mesmo usando os ganhos menores ($k = 1.0$ e $k = 3.0$), as letras ganham contornos escuros gigantes e o texto fica com um aspecto "sujo" e estranho. Esse teste prova que para imagens com detalhes muito pequenos, como textos e linhas finas, usar filtros menores (tipo 3×3 ou 5×5) dá um resultado muito melhor do que usar filtros grandes.

4.5 Cenário Extremo de Estresse ($m = 51$ e $k = 100.0$)

Para fechar os experimentos e ver o limite absoluto do algoritmo, rodamos um teste juntando o pior dos dois mundos na imagem de texto: uma máscara gigante de 51×51 combinada com um ganho absurdamente alto de 100.0. O resultado dessa combinação pode ser visto na Figura 7.

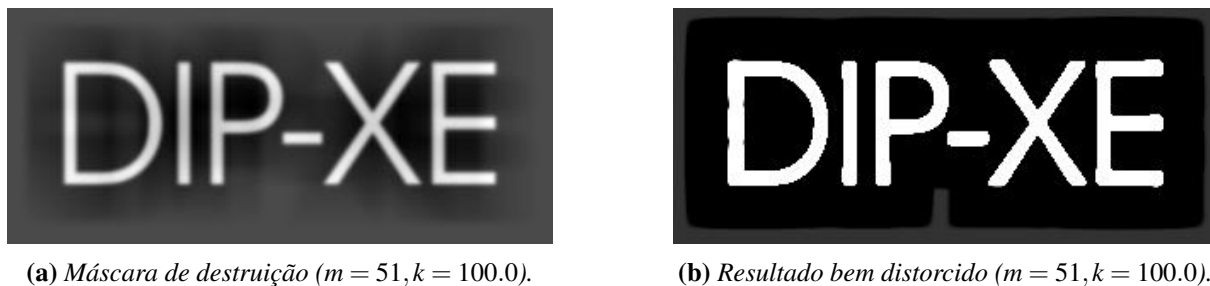


Figura 7. Teste de estresse máximo na imagem de texto usando os maiores parâmetros possíveis.

Análise: Como mostrado na Figura 7, o resultado foi a destruição completa da imagem. A máscara de tamanho 51 borrou tanto o texto original que ela acabou capturando o bloco inteiro onde as frases estavam escritas. Quando aplicamos o ganho de 100.0 em cima disso, o comando `np.clip` teve que cortar valores gigantescos. Isso fez com que o fundo branco ganhasse um quadrado cinza enorme e escuro ao redor das palavras, enquanto as letras estão com partes meio "comidas". Esse teste serve para isolar e provar de forma definitiva o comportamento limite do código em situações de erro ou abuso de parâmetros.

5 Conclusão

Fazer esse trabalho ajudou a entender na prática como funcionam os filtros de nitidez e como as nossas escolhas mudam o resultado visual da imagem. Conseguimos ver que colocar valores ruins nos parâmetros pode estragar completamente a foto: ou criando contornos grossos e esquisitos quando a máscara é grande demais ($m = 31$), ou gerando manchas e estourando as cores quando o ganho é muito alto ($k = 30$).

Fazer o teste de controle com $k = 0$ e tomar o cuidado de usar o tipo `float64` e a função `np.clip` evitou que o código desse erros de estouro de memória, o que é bem comum nesse tipo de algoritmo. O trabalho foi muito legal de programar e os testes deixaram os conceitos do livro bem claros. Agradecemos ao professor Franklin por passar esse projeto, foi top.